KrixGPT

An agentic AI assistant with tools, RAG, long-term memory, multi-provider LLM failover and Google sign-in

Project & Interview Guide

Rohit Rai | github.com/RohitRai19/KrixGPT | November 2026

Stack at a glance: Python 3.11 - FastAPI - LangGraph / LangChain 1.x - Groq (GPT-OSS) + Google Gemini - ChromaDB - SQLite / SQLAlchemy - Firebase Authentication - Tavily, Open-Meteo, Yahoo Finance - Docker - Hugging Face Spaces

1. The 60-second pitch

KrixGPT is a ChatGPT-style web application in which the model does not just answer from its weights: it decides when to call tools. It can search the live web, read documents the user uploads, remember facts about the user across conversations, do exact arithmetic, and fetch live weather and stock prices. It runs on multiple LLM providers with automatic failover, so a daily quota or an outage on one provider does not take the product down.

The backend is a FastAPI service that streams tokens to the browser over Server-Sent Events. The agent is a LangGraph state machine (model node and tool node in a loop) with a SQLite checkpointer, so every conversation is resumable. Users can chat instantly as guests, or sign in with Google (Firebase) to keep chats and memory across devices. The whole thing ships as a Docker image and runs for free on Hugging Face Spaces.

What I would highlight to an interviewer:

- **Agentic loop, not a prompt wrapper.** Tool selection, argument filling and multi-step reasoning are handled by the graph.
- **Reliability engineering around LLM APIs.** Provider abstraction, startup key validation, bounded retries, cross-provider fallback, friendly error surfaces.
- **Real data problems solved.** Person-scoped memory, guest-to-account migration, per-user ownership rules, auto-migrating schema.
- **Production hygiene.** Secrets only in environment, HttpOnly signed cookies, pinned dependencies, Docker, observability with LangSmith.

2. Architecture

Components and how a message flows

```
Browser (HTML/CSS/JS, SSE reader)
  | POST /chat/stream {message, thread_id, model, guest_id}
  v
FastAPI (app.py) --auth cookie--> auth.py (Firebase token verify, signed session)
  | save user msg (SQLite) . set thread + owner context for tools
  v
LangGraph agent (agent.py) [START] -> chatbot -> tools_condition -> tools -> chatbot ...
  | SqliteSaver checkpointer keyed by thread_id
  |-- providers.py: Groq GPT-OSS (default) | Gemini | with_fallbacks(...)
  |-- tools.py / extra_tools.py: web_search (Tavily), calculator, search_uploaded_documents
(Chroma RAG),
  | remember_this / recall_memory (SQLite), get_weather (Open-Meteo), get_stock_price
(yfinance)
  v
SSE events: {token} ... {title} {done} | {error} --> browser renders Markdown, updates sidebar
```

Request lifecycle, step by step

- The browser posts the message with its thread id, chosen model and a stable guest id. A session cookie is attached if signed in.
- FastAPI resolves the user (or guest), enforces conversation ownership (403 if the thread belongs to someone else), stores the user message and sets the thread/owner context the memory tools read.

- The LangGraph agent is fetched from a per-model cache. The chatbot node prepends the system prompt and calls the model with tools bound. If the response contains tool calls, the tools node executes them and control returns to the chatbot node; otherwise the graph ends.
- Only genuine assistant text chunks are streamed; tool-call JSON and tool outputs are filtered server-side so raw payloads never reach the UI.
- After the first exchange in a thread, a cheap model generates a 3-6 word title which is streamed as a separate event and persisted.
- Every step is checkpointed in SQLite, so the conversation can be resumed and traced in LangSmith.

Storage model

Store	What lives there	Why this store
SQLite via SQLAlchemy (data/chatbot_memory.db)	conversations (thread_id, title, user_id), chat_messages, long_term_memory (owner_id)	Zero-ops, file-based, perfect for a single-instance demo; the SQLAlchemy layer makes a move to Postgres a connection-string change.
SQLite via LangGraph SqliteSaver (data/langgraph_checkpoints.sqlite)	Graph state per thread (message history the model sees)	Built-in checkpointer; enables resumable threads and human-in-the-loop interrupts later.
ChromaDB (chroma_db/)	Embedded chunks of uploaded documents with thread_id and source metadata	Local persistent vector store with metadata filtering, no external service or key.
Browser localStorage	thread id per account, guest id, guest thread list, theme, selected model	Per-device conveniences that do not need the server.

3. Dependencies, versions and why

All versions are pinned in requirements.txt and were the exact versions used in development (Python 3.11.9).

Web layer

Package	Version	Role	Why this and not the alternative
fastapi	0.136.3	HTTP API, dependency injection for auth, templates	Async-native with first-class streaming responses and typed dependencies. Flask needs extra work for SSE and async; Django is heavier than a single-service app needs.
uvicorn	0.48.0	ASGI server	The standard server for FastAPI; supports streaming and reload in dev.
jinja2	3.1.6	Renders index.html with server state (Firebase config, current user, model list)	Lets the dropdown and auth state be rendered server-side from configuration instead of hard-coding in JS.
python-multipart	0.0.29	Parses document uploads	Required by FastAPI for multipart forms.
python-dotenv	1.2.2	Loads .env locally	Keeps secrets out of the repo; in production the same names come from the host's secret store.
itsdangerous	2.2.0	Signs the session cookie	Tamper-proof, time-limited cookie with no server-side session table; Flask uses the same library.

LLM and agent layer

Package	Version	Role	Why this and not the alternative
langchain / langchain-core	1.3.2 / 1.4.0 (1.7.6 resolved)	Message types, tool decorator, Runnable composition, with_fallbacks	Mature abstractions over many providers; one tool definition works with any model.
langgraph	1.2.2	The agent as a state graph (chatbot node, tool node, conditional edge)	Explicit control flow and checkpointing versus the opaque legacy AgentExecutor; needed for resumable threads and future interrupts.
langgraph-checkpoint-sqlite	3.1.0	Persists graph state per thread	File-based, zero infrastructure; swappable for Postgres checkpointer.
langchain-openai	1.7.4	Client for OpenAI-compatible APIs (Groq; also xAI)	Groq exposes an OpenAI-compatible endpoint, so one client covers several providers via base_url.
langchain-google-genai	4.2.4	Gemini chat models and gemini-embedding-001 embeddings	Native Gemini integration with tool calling and streaming; embeddings have a generous free quota.
langchain-text-splitters	1.1.2	RecursiveCharacterTextSplitter (900 chars, 150 overlap)	Keeps semantic units intact better than fixed-size splitting; overlap preserves context across chunk edges.
langchain-chroma / chromadb	1.1.0 / 1.5.9	Vector store for RAG with metadata filter by thread	Local, persistent, free; FAISS lacks persistence and metadata filtering out of the box, Pinecone needs an account and network.
langchain-tavily / tavily-python	0.2.18 / 0.7.24	Web search tool returning LLM-ready snippets	Purpose-built for agents (clean content, no HTML scraping), free tier of ~1000 searches a month.
langsmith (transitive)	0.14.17	Tracing of every run, tool call and fallback	Essential for debugging agent behaviour; enabled purely by environment variables.

Data, auth and live-data layer

Package	Version	Role	Why this and not the alternative
sqlalchemy	2.0.50	ORM and migrations for conversations, messages, memory	Database-agnostic; the demo runs on SQLite and production can run on Postgres unchanged.
pypdf / docx2txt	6.12.2 / 0.9	Text extraction for PDF and DOCX uploads	Pure-Python, light, no system dependencies (no poppler/libreoffice in the container).
firebase-admin / google-auth	7.7.0 / 2.62.1	Verify Google ID tokens from the browser	Firebase handles OAuth UI and account management for free; google-auth verifies tokens with only the project id, no service-account file.
yfinance	1.8.0	Stock quotes and stats	No API key; covers US, NSE/BSE and indices. Fails gracefully; an Alpha Vantage/Finnhub key can be swapped in for SLAs.
requests	2.34.2	Open-Meteo weather and geocoding calls	Open-Meteo is free with no key; requests keeps the tool dependency-light.
certifi	2026.10.23	CA bundle pinned via SSL_CERT_FILE	Avoids corporate/Windows SSL interception failures when calling Google APIs.

4. Key design decisions

LangGraph over a plain chain or the legacy AgentExecutor

A chain cannot loop. The agent needs: call model, maybe call tools, call model again. LangGraph makes that loop explicit (chatbot -> tools_condition -> tools -> chatbot), checkpoints state per thread, and later supports interrupt() for human approval. The legacy AgentExecutor hid this control flow and is not the recommended path in LangChain 1.x.

Server-Sent Events over WebSockets

Token streaming is one-directional server-to-client. SSE is plain HTTP, works through proxies and on Hugging Face without special configuration, and the browser's fetch + ReadableStream reads it with ~30 lines of JS. WebSockets would add connection management for no benefit.

A provider registry instead of hard-coding one model

providers.py maps model ids to providers, validates each key once at startup with a cheap /models call, hides providers whose keys are rejected, picks the default from the first working provider, and builds fallbacks. Groq (GPT-OSS 120B) is the default because its free tier allows thousands of requests a day; Gemini stays available as a second provider. Grok/xAI is pre-wired but off until a key is set.

Groq and xAI through one OpenAI-compatible client

Both expose the OpenAI chat-completions API. Using langchain-openai with a custom base_url means adding a provider is a dictionary entry, not a new integration.

Tools for structured/live data, RAG for text

Weather, stocks and (planned) MongoDB orders are exact lookups: a tool runs a precise query and returns facts. RAG is for uploaded documents where similarity search over text is the right operation. Mixing them up gives wrong numbers or stale results.

Memory scoped to a person, not a thread

Long-term memory rows carry an owner_id: the Firebase uid, or guest:<browser id> for guests. A fact learned in one chat is recalled in a new chat, and guest memories are re-homed to the account on sign-in. Uploaded documents stay per thread on purpose.

Guest-first with optional sign-in

Like ChatGPT: no login wall. Guests use unowned threads the browser remembers; signing in claims those threads and memories. Ownership is enforced server-side (403 for anyone else's thread), not just hidden in the UI.

Firebase for identity, own cookie for sessions

Firebase handles the Google OAuth popup and account storage for free. The backend verifies the ID token once and issues its own signed, HttpOnly, SameSite cookie, so the client never re-sends Firebase tokens and the server does not depend on Firebase per request.

SQLite + Chroma locally, with a clean exit to Postgres/managed vectors

For a single-instance demo on free hosting, file stores are the right trade-off: no services to run, no bills. SQLAlchemy and the LangGraph checkpointer interface make the Postgres move a configuration change when needed.

Server-rendered state in a JSON script block

The page needs the Firebase config, the current user and the model list. They are injected by Jinja into a <script type=application/json> block and parsed once. This keeps template syntax out of the JavaScript (editors and linters stay clean) and avoids XSS from naive interpolation.

Automatic chat titles with a cheap model

After the first exchange, the cheapest model of the default provider writes a 3-6 word title (one extra small request per conversation). A heuristic fallback strips greetings and filler so the sidebar never shows a raw prompt even if the title call fails.

5. Problems I hit and how I solved them

These are real issues from the development log, in the order they appeared. Interviewers usually care more about these than about the happy path.

Gemini 2.5 was rejected for new API keys (404 'no longer available to new users')

The tutorial-era model names were dead. I queried the models.list endpoint with my key, moved to the current catalog (gemini-3.8-flash and friends), and later generalised this into a model registry so model churn is a data change, not a code change.

The free-tier daily quota (20 requests per model) vanished in minutes

Two causes: the client retried 503s up to six times, each counting against quota, and tool calls cost two model requests per turn. Fixes: max_retries lowered to 1-2, titles moved to a different, cheaper model, Groq (thousands of free requests/day) made the default, and cross-provider failover so a 429 on one provider is retried on another instead of surfacing to the user.

503 'high demand' on one Gemini model made the first failover attempt useless

with_fallbacks raises the first error if every fallback fails, and my single fallback target was itself returning 503. I widened the fallback list: other models from the same provider first, then two models from each other provider.

A pasted API key turned out to be a Cursor service token

Both Groq and xAI rejected it. I added startup key validation: one request to the provider's /models endpoint; a 400/401/403 hides that provider and the default moves to the next working one. A bad key now produces a log warning, never a broken default for users.

Groq retired the Llama models I had listed

Discovered when listing the catalog with the real key. Replaced with GPT-OSS 120B/20B and Qwen 3.8 27B. Lesson: never trust remembered model names, query the catalog.

Chat titles came back empty on GPT-OSS

GPT-OSS is a reasoning model; with max_tokens=24 it spent the whole budget thinking and returned no text. Fixed with max_tokens=200 and reasoning_effort='low' for background jobs.

Memory did not carry over to a new chat

Memories were keyed by thread_id, and every new chat has a new thread. Added an owner_id column (auto-migrated and back-filled), scoped save/recall by uid or guest id, and re-homed guest memories on login. Verified end to end: save a name in chat A, recall it in chat B, and a different guest cannot see it.

Raw tool JSON appeared in the chat stream

LangGraph's messages stream includes tool-call chunks and ToolMessages. A should_stream_chunk filter passes only genuine assistant text, checking node name, message type, tool_calls and additional_kwargs.

Two processes ran the SQLite migration at once and crashed on 'duplicate column'

uvicorn --reload and a test client both called init_db() in the same instant. The ALTER TABLE is now wrapped to tolerate the duplicate-column error, so startup is idempotent.

Firebase Admin's verify_id_token demanded application credentials

Even verification needed a service-account file in that SDK version. We fall back to google.oauth2.id_token.verify_firebase_token, which checks the signature against Google's public certificates using only the project id, so no credentials file ships with the app.

Jinja placeholders inside the <script> tag broke the editor's JavaScript checker

Moved the server state into a <script type=application/json> block that is parsed at runtime. Zero diagnostics, and safer than interpolating into code.

Mobile layout, raw error dumps and tutorial branding in the original UI

Rebuilt the front end as a responsive app shell: off-canvas sidebar drawer, safe-area aware composer, Markdown rendering, dark/light theme, and error cards that translate 429/503/404/400 into plain-language guidance with technical details collapsed.

Working on the same files as another contributor reverted the template once

A file on disk silently lost the guest-mode changes while the commit still had them. Restored from git, re-applied the pending change, and treated git as the single source of truth from then on.

6. Security and reliability checklist

- Secrets only in environment variables (.env locally, host secrets in production); .env, data and vector stores are git-ignored and docker-ignored.
- Session: signed, HttpOnly, SameSite=Lax cookie with a 14-day max age; secure flag on HTTPS. Logout clears it server-side.
- Authorization enforced in the API, not the UI: every thread has an owner; guests may only touch unowned threads; others get 403.
- Calculator uses eval with an empty __builtins__ and an allow-list of math functions only.
- Tool outputs are the only thing the model sees from external systems; no raw database or shell access is exposed to the model.
- Bounded retries and cross-provider fallbacks so a provider incident degrades gracefully instead of failing the turn.
- Every external call (weather, stocks, provider validation) has a timeout and returns a readable message on failure.
- Known limitation to state honestly: tool context (thread/owner id) is a module-level variable, fine for one worker, to be replaced by contextvars or per-request tool binding before multi-worker scaling.

7. Deployment

The repo contains a Dockerfile (python:3.11-slim, non-root user, port 7860), a .dockerignore, a docker-compose.yml for local runs with persistent volumes, and a Hugging Face Space header in the README. Deployment target: Hugging Face Spaces, Docker SDK, free CPU tier (16 GB RAM, HTTPS, direct *.hf.space URL). Secrets are set in the Space settings; the Space hostname is added to Firebase's authorized domains.

Environment variables

```
GROQ_API_KEY, GOOGLE_API_KEY, TAVILY_API_KEY, SESSION_SECRET, optional XAI_API_KEY,
DEFAULT_MODEL, TITLE_MODEL, GENERATE_TITLES, LANGSMITH_TRACING, LANGSMITH_API_KEY,
LANGSMITH_PROJECT, FIREBASE_* overrides.
```

Free-tier trade-offs I can explain

- Disk resets on rebuild, so chats persist between visits but not across redeploys. Fix: Postgres for SQLAlchemy and the checkpointer, hosted vector store for Chroma.
- Space sleeps after ~48h idle; first visit wakes it in ~30 s. Fix: paid always-on hardware or a scheduled ping.
- Yahoo Finance is unofficial; the tool fails gracefully. Fix: a licensed market-data API when reliability matters.

8. What I would build next

- **MongoDB order tools.** Typed, user-scoped tools (get_order, list_orders, order_summary, search_products) with a read-only DB user; product text embedded for semantic search. Tools, not RAG, for structured data.
- **Human-in-the-loop.** LangGraph interrupt() before side-effect tools (cancel order, refund, address change); an approval card in the UI and a /chat/resume endpoint using Command(resume=...). The existing checkpointer already supports this.
- **MCP server.** Expose the same tools over the Model Context Protocol so desktop AI assistants and IDEs such as Cursor can use them; a thin wrapper, not a rewrite.
- **Per-request tool context** via contextvars, Postgres storage, rate limiting per user, and evaluation with LangSmith datasets.

9. Likely interview questions with answers

Q. Why is this 'agentic' and not just a chatbot?

The model decides at each step whether to answer or call a tool, the graph executes the tool and feeds the result back, and this loops until the model produces a final answer. The control flow is explicit in LangGraph and every step is checkpointed and traceable in LangSmith.

Q. How does streaming work end to end?

agent.stream(stream_mode='messages') yields message chunks as the model generates. The FastAPI endpoint filters to genuine assistant text, wraps each piece as an SSE 'data:' event, and the browser reads the response body with a ReadableStream, appending tokens live and rendering Markdown when 'done' arrives.

Q. What happens if Gemini hits its daily quota mid-conversation?

The 429 is caught by the fallback wrapper, the same request runs on the next configured model (another Gemini model, then Groq), and the user sees a normal answer. The next message tries the user's chosen model again. If everything fails, the UI shows a plain-language error card with technical details collapsed.

Q. Why not use RAG for the order data you plan to add?

Orders are structured and change constantly. 'How many orders last month' needs an exact filter and a count. Embedding similarity cannot do arithmetic or stay fresh. Tools run precise queries; RAG is reserved for unstructured text like product descriptions or policies.

Q. How do you keep one user from reading another's chats?

Every conversation row has a user_id. Each API route resolves the caller from the signed cookie (or treats them as a guest), looks up the thread's owner and returns 403 on mismatch. Guests can only access unowned threads, and the browser only knows the thread ids it created.

Q. How does guest memory work without an account?

The browser generates a UUID once and sends it with every message. Memories are saved under guest:<uuid>. On sign-in that uuid is sent with the ID token and the server re-homes those memories (and the guest's threads) to the Firebase uid.

Q. Why Groq as default rather than OpenAI or Gemini?

Cost and limits for a public demo: Groq's free tier allows thousands of requests per day on strong open models with tool calling and very low latency. Gemini's free tier is 20 requests per model per day, which an interviewer could exhaust in one session. The design keeps all of them pluggable.

Q. What would break first under real traffic?

SQLite and the module-level tool context under multiple workers, and the single Chroma directory. The fixes are known: Postgres for both SQLAlchemy and the LangGraph checkpointer, contextvars or per-request tool binding, and a hosted vector store.

Q. How do you test an LLM app?

Deterministic parts with the FastAPI TestClient and direct tool calls (auth gating, ownership rules, migrations, parsers). LLM-dependent paths with short, cheap end-to-end runs that assert on structure (a tool was called, a title event arrived) rather than exact wording, plus LangSmith traces for inspection.

Q. What did you learn from this project?

That most of the engineering in an LLM product is around the model, not in the prompt: quotas, retries, provider churn, identity, data scoping and honest error handling. And to query the real catalog and real APIs instead of trusting remembered model names or key formats.